High-Level Design (HLD) Document of Communication Module

Emergency Opportunistic Relay System (EORS)

Table of Contents

1. Executive Summary

Problem Statement

In rural, disaster-prone, and low-connectivity environments, individuals are unable to access emergency services due to the absence of telecom infrastructure. Critical situations (e.g., pregnancy complications, cardiac arrest, accidents) require instant communication, not delayed synchronization. The "Golden Hour" of medical response is often lost simply because a distress signal cannot traverse a few miles to the nearest active cell tower.

Proposed Solution

The Emergency Opportunistic Relay System (EORS) enables real-time emergency signal transmission through nearby mobile devices using peer-to-peer relay mechanisms (Bluetooth Low Energy + WiFi Direct). Devices with internet act as dynamic gateways to cloud emergency servers. EORS does not replace telecom infrastructure. Instead, it bridges connectivity gaps using opportunistic distributed networking, effectively turning every active smartphone into a temporary, secure router for distress signals.

2. System Overview

2.1 Project Background

Traditional emergency apps rely on cellular or WiFi networks. When these fail, communication collapses. Disaster networks and military-grade mesh systems demonstrate that peer-to-peer relays can function without central towers. EORS adapts this concept into a scalable civilian mobile application, ensuring that a localized mesh can organically form and dissolve based on physical device proximity.

2.2 Key Objectives

- Enable near real-time emergency communication without direct connectivity.
- Preserve user privacy through zero-knowledge architectures.
- Minimize battery impact on intermediary relay devices.
- Provide scalable backend dispatch to handle sudden regional surges.
- Maintain regulatory compliance regarding medical and location data.

2.3 Scope and Boundaries

In Scope:

- Emergency packet broadcast.
- Opportunistic relay.
- Secure encryption.
- Cloud dispatch integration.

Out of Scope:

- Guaranteed communication in absolute isolation (e.g., a single person in the middle of the ocean).
- Hardware-based satellite systems.
- Replacement of telecom networks.

3. Architecture Principles

3.1 Design Philosophy

- **Offline-first:** Core logic executes without requiring a server handshake.
- **Security-by-design:** All payloads are treated as hostile and sensitive.
- **Event-driven:** System reacts to BLE triggers instantly.
- **Modular microservices:** Independent backend scaling.
- **Fail-safe design:** App degrades gracefully if specific hardware sensors fail.

3.2 Architectural Patterns

- **Peer-to-Peer Mesh:** For local topology formation.
- **Store-and-Forward:** Packets are held locally until a hop is available.
- **Microservices Backend:** Handling decryption, dispatch, and logging.
- **Event Streaming & CQRS (Command Query Responsibility Segregation):** Separating the high-velocity writes of emergency broadcasts from backend analytical queries.

3.3 Critical Design Decisions

- Use BLE for low-energy discovery to maintain battery life.
- Use WiFi Direct for high-bandwidth fallback if payload size exceeds BLE limits.
- End-to-end encryption mandatory to protect intermediary liability.
- Stateless cloud services for rapid auto-scaling.

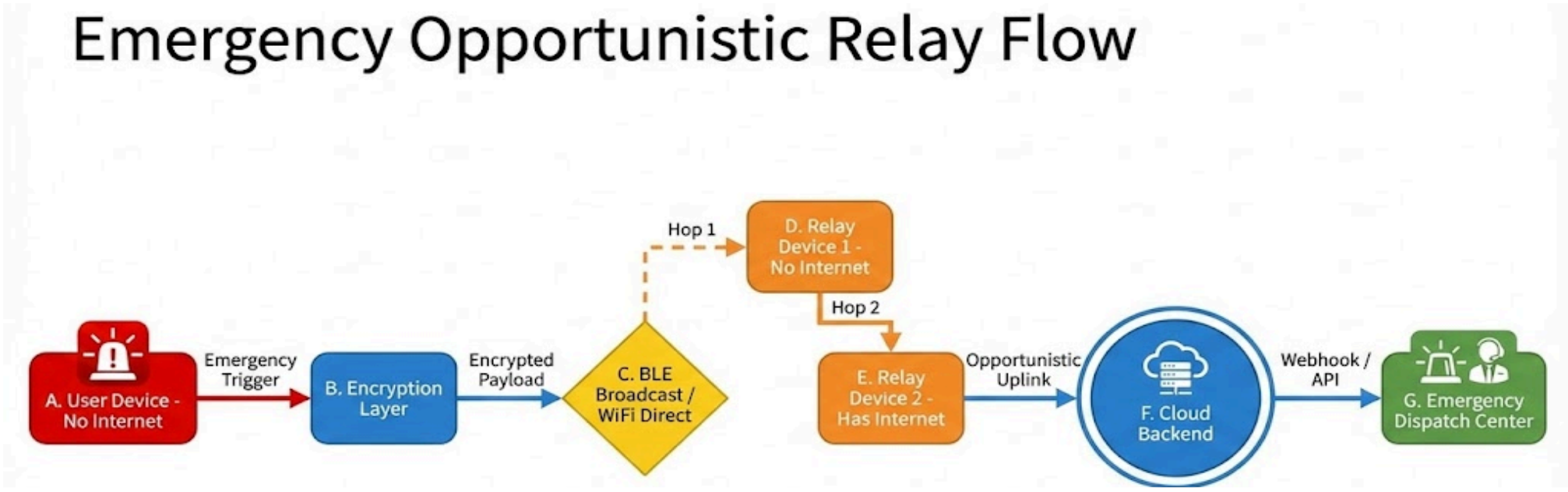
4. Technology Stack

Layer	Technology	Justification
Mobile Layer	Flutter, Native BLE APIs, WiFi Direct, Isar Database	Native APIs ensure deep OS access for background networking. Isar provides high-speed, ACID-compliant local storage.
Backend Layer	FastAPI, PostgreSQL, Redis, Cloud Run	FastAPI ensures low-latency ML and routing execution. Cloud Run enables zero-to-hero serverless scaling.
Security	TLS 1.3, JWT, ECC key pairs, AES-256 encryption	ECC provides strong security with small key sizes (ideal for BLE payloads).

5. System Architecture

5.1 High-Level Architecture Diagram

```
graph TD
  A[User Device - No Internet] -->|Emergency Trigger| B(Encryption Layer)
  B -->|Encrypted Payload| C{BLE Broadcast / WiFi Direct}
  C -->|Hop 1| D[Relay Device 1 - No Internet]
  D -->|Hop 2| E[Relay Device 2 - Has Internet]
  E -->|Opportunistic Uplink| F((Cloud Backend))
  F -->|Webhook / API| G[Emergency Dispatch Center]
```



(Diagram represents the flow: User Device (No Internet) -> Emergency Trigger -> Encryption Layer -> BLE Broadcast / WiFi Direct -> Relay Device -> Cloud Backend -> Emergency Dispatch Center).

5.2 Component Overview

- **Emergency UI Module:** One-touch SOS interface bypassing standard app navigation.
- **Network Monitor:** Continuously polls for cellular/WiFi states.
- **Relay Engine:** Manages duty-cycled advertising and scanning.
- **Encryption Manager:** Handles asymmetric key signing and symmetric payload encryption.
- **Local Storage:** Temporary holding area for orphaned packets.
- **Backend API Gateway & Dispatch Service:** Receives, decrypts, and routes the signal to local authorities.

6. Data Architecture

6.1 Database Strategy (Optimized)

- **Mobile Edge:**
  - Encrypted local storage (AES-256).
  - Temporary queue table for pending outgoing hops.
  - Relay attempt logs (purged upon successful sync).
- **Cloud Backend:**
  - Emergency events table (immutable ledger).
  - Device registry table (Public keys).
  - Relay audit logs & Geolocation index (PostGIS).

6.2 Data Flow Design & Payload Structure

To transmit via BLE, the packet must be incredibly small.

- **Emergency Packet Structure:** Device ID, Timestamp, Encrypted payload, Digital signature.
- **Data Flow Sequence:**
  1. Packet created by Victim Device.
  2. Encrypted using Backend Public Key.
  3. Broadcast via BLE Advertising Data.
  4. Relay device verifies packet format (drops malformed data).
  5. Forwarded to backend upon internet connection.
  6. Backend validates cryptographic signature.
  7. Dispatch initiated to EMS.

7. Security Architecture

Because devices are acting as "blind mules" for other people's data, security is paramount.

- **Public key infrastructure (PKI):** Ensures only the dispatch server can read the medical payload.
- **Certificate-based device identity:** Prevents rogue devices from flooding the network with fake SOS signals.
- **Zero-knowledge relay:** Intermediary devices cannot view, alter, or analyze the packet contents.
- **Tamper detection:** Digital signatures invalidate the packet if a single byte is altered in transit.
- **Rate limiting & DDoS protection:** Backend drops excessive identical UUIDs.

8. Deployment Architecture

8.1 Cloud Infrastructure

- **API Gateway & Load Balancer:** Front-facing ingress points.
- **Cloud Run containers:** Running FastAPI microservices for stateless processing.
- **PostgreSQL managed service & Redis cache:** For data persistence and high-speed memory access.
- **Monitoring:** Prometheus/Grafana for system health.

8.2 Cloud Run Scaling Considerations

- Auto-scaling based on incoming request load (critical during a regional disaster).
- Stateless containers ensure no data is lost if an instance crashes.
- Cold start mitigation via minimum provisioned instances.
- Regional deployment for low latency dispatch execution.

9. Risk Management

Risk	Impact	Deep Mitigation Strategy
Low user density	Relay failure	Multi-hop routing (packet jumps up to 5 times). Integration with static community hubs.
Battery drain	User dissatisfaction	Adaptive scanning algorithms that respect OS battery limits.
Security breach	Data exposure	Strict end-to-end encryption. No symmetric keys stored on-device.
False emergency spam	Resource waste	Cryptographic signature validation. Devices flagged for spam are blacklisted at the gateway.
Network Flooding	Bandwidth exhaustion	Implementation of UUID-based packet deduplication.

10. Deep Problem Analysis with Deepest Solutions

Problem 1: Absolute Network Absence

- **Deep Analysis:** Without RF infrastructure, communication requires an alternate propagation path.
- **Deep Solution:** Opportunistic mesh relay leveraging nearby devices.
- **Implementation Approach:** Utilizing BLE beacon advertisement to perform a peer discovery handshake. Once discovered, an encrypted session key exchange occurs before forwarding to an internet gateway.

Problem 2: Latency Constraints

- **Deep Analysis:** Emergency communication must be <5 seconds to be effective for dispatch.
- **Deep Solution:** Priority interrupt routing + multi-path broadcast.
- **Implementation:** The device broadcasts to multiple peers simultaneously rather than searching for one optimal path. The first successful gateway forwards the packet. Duplicate suppression via UUID ensures the backend doesn't trigger 5 ambulances for the same event.

Problem 3: Security & Privacy (Liability)

- **Deep Analysis:** Relay devices must not access medical data to comply with privacy laws (e.g., HIPAA).
- **Deep Solution:** End-to-end encryption + backend public key encryption.
- **Implementation:** The origin device encrypts the payload using the backend public key. The relay simply forwards an opaque binary packet, acting purely as a transport layer.

Problem 4: Scalability During Crises

- **Deep Analysis:** A large user base experiencing a regional disaster (e.g., earthquake) increases emergency volume exponentially.
- **Deep Solution:** Cloud-native microservices + horizontal scaling.
- **Implementation:** Utilizing stateless services and event queue buffering (Redis/Kafka). Auto-scaling policies instantly spin up new containers to handle the traffic spike.

Problem 5: Power Efficiency (The Bystander Problem)

- **Deep Analysis:** Continuous background scanning drains the battery, causing healthy users to uninstall the app.
- **Deep Solution:** Duty-cycled scanning + adaptive intensity.
- **Implementation:** The app performs a 5-second scan every 60 seconds during normal operation. A full, continuous scan is only activated when a user explicitly triggers emergency mode.

11. Glossary

- **BLE** – Bluetooth Low Energy.
- **EORS** – Emergency Opportunistic Relay System.
- **PKI** – Public Key Infrastructure.
- **CQRS** – Command Query Responsibility Segregation.

Conclusion

The Emergency Opportunistic Relay System is a distributed, privacy-preserving, cloud-integrated emergency communication platform designed for intermittent connectivity environments. It rigorously balances real-time life-saving requirements with the physical constraints of wireless networking and device battery limits, delivering a technically feasible and scalable emergency solution.